

Malware Similarity Trends and Gaps Analysis

Midterm Report

Anmol Agarwal	Anmol.Agarwal@utdallas.edu
Gina Fossati	Gina.Fossati@utdallas.edu
Jennifer Ward	Jennifer.Ward@utdallas.edu

April 3, 2020

Project Summary

Description

People have been trying to address malware similarity both academically and operationally for decades, with no clear community consensus on techniques or even which malware features are most relevant. We need to look at the problem differently. Recent efforts at MITRE have proposed a new methodology focusing on the fingerprints left behind by the Malware Development Life Cycle (MDLC). Malware is continuously changing, and today's malware is characterized as yesterday's malware plus more. Since malware is software, it is important to understand its development life cycle. The life cycle can be broken down into eight phases: requirements, design, code, build, configure, package, collection, and analysis. We will focus on the first six phases because collection and analysis are not direct properties of the malware and its author. We will consider the process of a software developer in each of the phases and determine what we can extract from looking at the end product. Some additional things we will assess include the knowledge or skill level of the malware's author and links between phases of different malware that are similar due to unique choices in development and operation. In addition to the information found in each phase of the life cycle, we must also consider obfuscation in our analysis, such as heavily obfuscated namespaces, source code obfuscated by multiple layers, and control flow obfuscation where a single function could be broken into multiple functions and indirect calling. Our goal is to create a prototype that automates the analysis of features found in the phases of the Malware Development Life Cycle (MDLC) as seen in the final product or malware executable. From this analysis, we will attempt to identify similarities in malware.

Required Skills

- Basic understanding of software engineering (e.g. waterfall, spiral, agile).
- Basic understanding of software development (e.g. how to compile a program).

- Comfortable reading programs written in a scripting language (e.g. Python, Powershell, PHP).
- Comfortable reading programs in a compiled language (e.g. C/C++, Delphi, Go, Rust).
- Comfortable reading disassembled x86 code.

Team Responsibilities

- Review the Malware Development Life Cycle (MDLC) and understand the phases.
 - Focus on adversary activities and how the malware is made (languages used, data structures, compiler linking options, etc.) to determine similarities in malware.
- Set up a malware analysis environment.
- Develop implementations of an exfiltration tool in various languages using different packaging and configuration options.
- Determine features found in each layer of the MDLC to extract from a malware executable.
- Develop a prototype of an automated tool to extract these features in order to compare similarities in malware.

Total Budget

The team will spend a total of ten hours combined per week on the project.

Motivation

As malware are addressed, it is important to have a clear community consensus on how to define and measure similarity. We are interested in becoming more familiar with malware features, analyzing malware, and detecting similarities and differences to classify malware “families”. All group members have experience with software engineering, methodologies, and development and are comfortable reading both scripting and compiled languages (e.g., Python, C/C++, Go). Additionally, most of our team members have completed coursework involving MIPS assembly, and some have experience with x86. We are excited to apply these skills to malware analysis and classification.

Literature Review

1 Comparative Analysis of Feature Extraction Methods of Malware Detection

This paper analyzes the general framework of malware detection systems and discusses the strengths and weaknesses of each method. Malware, including new, unfamiliar variants, tend to exhibit similar patterns in structure, which allows malware detection systems to detect unfamiliar

malware through feature extraction. Current methods are based on two complementary approaches: signature-based detection and heuristic based approaches. The signature-based approach relies on the identification of unique string patterns in the binary code through a static technique, but it cannot detect zero-day attacks and attackers can easily work around this detection system through obfuscation. The heuristic approach is based on rules determined by experts by dynamically analyzing behavior to see when it deviates from normal behavior which effectively detects unfamiliar malware but tends to raise more false alarms and requires more resources and time. Others use a hybrid approach which integrates static and dynamic analysis through feature extraction. Feature extraction is performed to reduce a large input or dataset into an organized, more manageable subset of information that requires less processing and computational overhead. Therefore, the feature extraction method used affects the performance of the system through efficiency, robustness, and accuracy. They conclude the paper with a performance overview of malware detection systems based on features. The efficiency and robustness of the system is defined by high accuracy, high True Positive Rate, and low False Positive Rate.

2 Malware Images: Visualization and Automatic Classification

This paper describes a simple technique used to visualize and classify malware using image processing techniques. When malware is visualized as gray-scale images, those belonging to the same family tend to exhibit a similar layout and texture. This is most likely a result of the reuse of old malware binaries to create new variants. Due to this visual similarity, classification can be accomplished using standard image features rather than using code disassembly or execution as done in static and dynamic code analysis. This approach uses visual texture to automate classification by recognizing repeated patterns in the images through texture feature extraction. A common approach to feature extraction is accomplished through analyzing the frequency content of a texture block by dividing the frequency domain into rings and wedges such that features are found for those areas. They used GIST to compute texture features, which uses a wavelet decomposition of an image.

3 Novel Feature Extraction, Selection and Fusion for Effective Malware Family Classification

In this paper, a machine learning based system is proposed that assigns malware to their appropriate malware families. A Microsoft malware dataset released to Kaggle was used for training and testing. In this approach, malware features are extracted from hex and assembly views. Hex dumps are used to extract metadata and entropy. The malware files are also disassembled to determine features such as frequencies of the following symbols: -, +, *,], [, ?, @, Operation Code, Register, and API. If the frequencies of the symbols are high, then that is

indicative that the malware file is trying to evade detection. For classification, a Random Forest Algorithm was used to determine the importance of each feature category based on the mean decrease impurity. Each feature was ranked based on their Gini Impurity Scores. The feature that contributed most to the impurity was the metadata (size of the file, address of first bytes technique). The XGBoost algorithm was used with bagging and feature optimization to determine the feature categories and accuracy and LogLoss for the training and testing sets. With XGBoost, in the Hex Dump file, the features that had the highest 5CV accuracy were ENT (entropy) and 1G (1-gram features that represent byte frequency) with a 0.9862 and 0.9808 accuracy respectively. In the disassembled file the features that had the highest CV-5 accuracy were MISC (95 manually chosen keywords that could be extracted from the interpretation from IDA from the code and dll which shows the number of imported DLLs) and OPC (operation codes) with a 0.9917 accuracy and a 0.9907 accuracy respectively.

4 Differentiating Malware from Cleanware Using Behavioural Analysis

In this paper, malware features were extracted by examining the behavior of API calls in the malware files. Files are executed one by one and then execution logs are collected for each file. Runtime behavior was recorded for the binaries of the malware and cleanware. Then, trace reports using a tool HookMe were generated; these trace reports log various events, such as what API calls the binary file makes. A list of strings and string frequencies for each trace report is created, and then feature vectors are created using the original file's string frequencies. Some examples of string features extracted from trace reports are "Kernel32.dll" and "DontUseDNSLoadBalancing".

Once the string features were extracted, files were classified using the Weka library. Malware was classified using the built in Weka classifiers for classification such as Support Vector Machines (SVM), Random Forest, Instance- Based Classifier IB1, and Decision Table. Each file is also tested with k-fold cross validation. In this approach, malware and cleanware were classified and malware was differentiated from cleanware with a 97.4% accuracy.

5 A Lifecycle Based Approach for Malware Analysis

This paper uses the twelve stage lifecycle of behavioral activity to identify malware based on its behavior. This technique does not use signature based detection like many other methods. The lifecycle method breaks down into twelve classes: inbound scan, inbound attack, spontaneous attack, client-side exploit, egg download, device infection, local reconnaissance, network surveillance, C&C communications, peer coordination, attack preparation, and malicious outbound propagation. This method has favorable false positive, false negative, and accuracy compared to other techniques.

Progress

Overview

The main goal of this project is to develop an automated tool that analyzes malware and outputs useful information about the malware. To achieve this, we have researched the malware development life cycle and each written our own malware in various languages based on provided requirements for an exfiltration tool. We will compare the final product of our malware tools, in the form of Windows executables, to find similarities and differences evident from each phase of the MDLC. With this knowledge, we will have a basis to begin developing our prototype to extract these features automatically from a piece of malware.

Detailed Plan

Work completed

The team has developed variations of an exfiltration tool that can extract files specified by the attacker from a victim's machine, obfuscate these files, and send them to an IP address or FQDN through an HTTP Post command. The tool comes with an additional deobfuscation tool or option so that once the adversary retrieves these files from the IP address or FQDN, the exfiltrated files are deobfuscated. The team has developed six exfiltration tools, in which each team member developed two implementations in two different languages. These implementations consist of three variations of an exfiltration tool in Python from all different authors, two variations written in Java from different authors, and one exfiltration tool written in Go. Furthermore, the team has been able to set-up and remote access to two computers to be used as a malware analysis environment. One computer has a REMnux Virtual Machine installed that will assist the team with malware analysis.

Work in Progress

We are working on finding obfuscation techniques to obfuscate the developed malware using various packaging methods and tools. Specifically, we are working to package all of the exfiltration tools as standalone Windows executables with various configuration options such that an attacker can directly run the executable on the victim's machine. This specific goal is important because malware analysts typically only work with the final product or self-contained executable version of the malware. Once this task is completed, we will determine what features evident in each phase of the MDLC can be found in each of the resulting executables. Our goal is to then develop a prototype of an automated tool that will accomplish this task when analyzing malware. The results of this analysis will provide the foundation for identifying similarities in malware, specifically features common to an author or group.

Challenges

It is difficult to detect common features among malware when the malware is obfuscated by its author or authors. An author or group of authors of malware may purposefully make variations in implementation language, configuration options, and other features when writing each new piece of malware to prevent this type of analysis from proving effective. Furthermore, some key “fingerprints” found in a certain phase of the MDLC may not be retrievable in the resulting executable that will be analyzed.

Organization

Plans

The group will meet virtually once a week and communicate throughout the week through a messaging application to ensure progress on the project. The team will use effective time management and communication to ensure that the project continues to progress on schedule.

Deliverables

- Proposal & Literature Review
- Implementations of Exfiltration Tool
- Progress Report & Presentation
- Final Automated Tool Prototype
- Final Report & Presentation

Schedule

February 7: Literature Review
February 14: Proposal Due
February 14: Understand phases of Malware Development Life Cycle and develop plan
February 21: Set up Malware Analysis Environment
February 21: Start Malware implementations
February 28: Finish 1st Malware implementation
March 6: Finish 2nd Malware implementation
March 13: Finish 2nd Malware implementation
Spring Break: Research packaging/obfuscating
Brainstorm features from malware that indicate malware similarities
Finalize components from MDLC that will be used in analysis
April 3: Midterm Report Due
April 10: Develop analysis tool

April 17: Develop analysis tool
April 24: Finish analysis tool, Review Final Report
May 1: Final Deliverables Due

Budget

There are no monetary costs associated with this project. The team will meet approximately ten hours total per week and will have weekly status update meetings.

Broader Impact

We will develop a prototype of an automated technique to identify similar characteristics in malware based on attributes found in different phases of the Malware Development Life Cycle. This will provide insight into how malware is created and common adversarial techniques which can be used to detect the authors or operators of new malware and how it is different from previous malware.

Biographical Sketches of Investigators

Anmol Agarwal

Anmol Agarwal holds a Bachelor's degree in Software Engineering with a specialization in information assurance and is currently pursuing her Master's degree in Computer Science with a specialization in information assurance. She is also an NSF CyberCorps Scholarship for Service recipient. She has experience in software development, project management, virtual machines, and information security. Anmol has experience with solving CTF (Capture the Flag) challenges, such as challenges in web security and forensics. She has completed coursework in computer networks, digital forensics, information security, artificial intelligence, operating systems, and data and applications security. She is currently taking machine learning and cloud security. In her free time, Anmol is an active member of Women in CyberSecurity and the Computer Security Group.

Gina Fossati

Gina Fossati graduated with her Bachelor's degree in Computer Science in May 2019, and she is currently pursuing her Master's degree in Computer Science in the Information Assurance Track. She is also an NSF CyberCorps Scholarship for Service recipient. She has completed coursework in information security, digital forensics, IOT security, language based security, operating systems, computer networks, and machine learning. She is currently taking data and application security, web programming languages, and advanced algorithms.

Jennifer Ward

Jennifer Ward graduated with her Bachelor of Science in Computer Engineering in December 2019 and is now pursuing her Master of Science in Computer Science with a specialization in information assurance. She is also an NSF CyberCorps Scholarship for Service recipient. She has experience in analyzing data, software development, and practicing and examining security attacks and vulnerabilities. She has completed coursework in systems security and malicious code analysis, computer networks, computer architecture, operating systems, software engineering, Linux operating system security, and embedded microprocessor systems. She is also currently taking data and application security and network security.

References

- [1] Ranveer, Smita, and Swapnaja Hiray. "Comparative Analysis of Feature Extraction Methods of Malware Detection." *International Journal of Computer Applications*, vol. 120, no. 5, June 2015, pdfs.semanticscholar.org/8ec8/858a25ce2fb76388d40ad07878b0ec79f580.pdf.
- [2] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath. 2011. Malware images: visualization and automatic classification. In *Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec '11)*. Association for Computing Machinery, New York, NY, USA, Article 4, 1–7. DOI:<https://doi.org/10.1145/2016904.2016908>
- [3] M. Ahmadi, D. Ulyanov, S. Semenov, M. Trofimov, G. Giacinto. March 2016. Novel Feature Extraction, Selection and Fusion for Effective Malware Family Classification. In *CODASPY '16: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy*. Association for Computing Machinery, New York, NY, USA, pages 183 - 194.
- [4] R. Tian, L.M. Batten, R. Islam, and S.C. Versteeg, "Differentiating Malware from Cleanware Using Behavioural Analysis," in *Proc. of the 5th IEEE Int. Conf on Malicious and Unwanted Software (Malware'10)*, Nancy, France, pp. 23–30, October 2010
- [5] S. K. Pandey and B. M. Mehtre, "A Lifecycle Based Approach for Malware Analysis," 2014 Fourth International Conference on Communication Systems and Network Technologies, Bhopal, 2014, pp. 767-771.